TransMamba-MoE: A Working Draft for a Sequence-Level Hybrid Transformer–Mamba Architecture with Fine-Grained Mixture of Experts

Mathematical Foundations, Notation Derivation,
and Resolution of Three Core Design Problems

Kiprop Yego

Independent Research

March 21, 2026
(Working Draft v1.0 — Pre-Implementation)

Abstract

Let $H \in \mathbb{R}^{T\times d}$ denote a sequence of T token embeddings of dimension d, and let $\ell \in \{1,\dots,L\}$ index the layers of a language model. TransMamba [Li et al., 2026] partitions H at a scheduled *TransPoint* $\mathcal{P}_\ell \in \{0,\dots,T\}$, routing tokens $t < \mathcal{P}_\ell$ through a Transformer attention mechanism and tokens $t \geq \mathcal{P}_\ell$ through a Mamba State Space Model (SSM) recurrence, with both mechanisms sharing a single weight matrix $\mathbf{W} \in \mathbb{R}^{d\times d}$ via the algebraic duality $Q \leftrightarrow C$, $K \leftrightarrow B$, $V \leftrightarrow x$. The present document proposes **TransMamba-MoE**, an extension of TransMamba that incorporates a DeepSeekMoE-style [Dai et al., 2024] fine-grained Mixture of Experts (MoE) at three distinct sites within each layer: (i) the shared weight matrix pool $\{\mathbf{W}^{(i)}\}_{i=1}^{mN_e}$ (the *W-MoE*), (ii) the SSM projection layers for Δ, A, and Z (the *SSM-MoE*), and (iii) the feed-forward sublayer (the *FFN-MoE*). We derive all notation explicitly from first principles, state and solve three design problems identified in the intersection of MoE and TransMamba: **Problem 1** (the expert-conditioned Memory Converter), **Problem 2** (mechanism-aware load balancing), and **Problem 3** (training stability under coupled TransPoint and routing dynamics). For each problem we provide a mathematically rigorous solution, state theorems characterising the solution's correctness, and describe the resulting implementation contract for the coding phase.

Contents

1 Motivation and Scope of this Draft **4**

2 Notation Derivation **4**

2.1 Sequence and Dimension Indices 4

2.2 Input and Hidden State Tensors 4

2.3 TransPoint Scheduling . . . 5
2.4 Shared Weight Matrix and Its Dual-Form Role . . . 5
2.5 Attention Mechanism Tensors . . . 5
2.6 SSM Mechanism Tensors . . . 6
2.7 Mixture-of-Experts Notation . . . 6
2.8 Loss Function Components . . . 7

3 Background: TransMamba and DeepSeekMoE 7
3.1 TransMamba: The Dual-Form Architecture . . . 7
3.2 DeepSeekMoE: Fine-Grained Shared-Expert Isolation . . . 8

4 The W-MoE: Expert Pool on the Shared Weight Matrix 9
4.1 Motivation and the Coupling Constraint . . . 9
4.2 W-MoE Router . . . 9
4.3 W-MoE Forward Pass . . . 10

5 The SSM-MoE: Expert Pool on SSM Projection Layers 10
5.1 Motivation . . . 10
5.2 SSM-MoE Expert Pools . . . 10
5.3 Shared SSM Router . . . 11
5.4 SSM-MoE Forward Pass . . . 11

6 The FFN-MoE: Expert Pool on the Feed-Forward Sublayer 11
6.1 Architecture . . . 11

7 Design Problem 1: The Expert-Conditioned Memory Converter 12
7.1 Statement of the Problem . . . 12
7.2 Derivation of the Expert-Conditioned Memory Converter . . . 12

8 Design Problem 2: Mechanism-Aware Load Balancing 13
8.1 Statement of the Problem . . . 13
8.2 Mechanism-Aware Load Balancing: Formulation . . . 14
8.3 Setting α_a and α_s . . . 15
8.4 Full Training Loss . . . 15

9 Design Problem 3: Training Stability 15
9.1 Statement of the Problem . . . 15
9.2 Analysis of the Instability Sources . . . 15
9.3 Solution: Three-Component Stability Protocol . . . 16

10 The Complete TransMamba-MoE Layer 17
10.1 Full Forward Pass . . . 17
10.2 Full Algorithm . . . 19

11 Parameter Count and FLOPs Analysis 20
11.1 Parameter Count . . . 20
11.2 FLOPs Per Layer . . . 20

12 Implementation Contract **20**
12.1 Data Structures 21
12.2 Critical Implementation Invariants 21
12.3 Suggested Hyperparameters for First Run 22

13 Open Theoretical Questions **22**

14 Conclusion **23**

1 Motivation and Scope of this Draft

This document is a *working mathematical draft* whose purpose is to lay the complete theoretical foundation for TransMamba-MoE before a single line of code is written. The philosophy is that every design decision made here should be traceable to a specific equation or theorem in this document, so that the implementation is a transcription of mathematics rather than a sequence of ad-hoc engineering choices.

The draft is structured as follows. Section 2 derives the full notation table from first principles, defining every symbol before it is used. Section 3 restates the essential mathematics of TransMamba [Li et al., 2026] and DeepSeekMoE [Dai et al., 2024] in this unified notation. Section 4 develops the W-MoE component. Section 5 develops the SSM-MoE component. Section 6 develops the FFN-MoE component. Section 7 solves Design Problem 1 (expert-conditioned Memory Converter). Section 8 solves Design Problem 2 (mechanism-aware load balancing). Section 9 solves Design Problem 3 (training stability). Section 10 assembles the complete TransMamba-MoE layer. Section 11 analyses parameter counts and FLOPs. Section 12 provides the implementation contract.

2 Notation Derivation

We derive every symbol used in this paper from semantic first principles, grouping symbols by the conceptual domain they inhabit. No symbol is used without prior definition in this section.

2.1 Sequence and Dimension Indices

$T \in \mathbb{N}$ The total number of tokens in the input sequence. In the experiments of Li et al. [2026] this is $T = 8{,}192$.

$d \in \mathbb{N}$ The *hidden (embedding) dimension*, the size of every token's representation vector at any point inside the model. In the 1.5B TransMamba of Li et al. [2026], $d = 1{,}536$.

$N \in \mathbb{N}$ The *SSM state dimension*, the size of the hidden state h_k maintained by the Mamba recurrence. In Li et al. [2026], $N = 1{,}536$.

$L \in \mathbb{N}$ The total number of stacked layers. In the 1.5B model of Li et al. [2026], $L = 64$.

$t \in \{0, \ldots, T-1\}$ A *token position index*, ranging over the full sequence.

$\ell \in \{1, \ldots, L\}$ A *layer index*.

$d_{\text{ffn}} \in \mathbb{N}$ The feed-forward sublayer intermediate dimension. In Li et al. [2026], $d_{\text{ffn}} = 4{,}096$.

2.2 Input and Hidden State Tensors

$H \in \mathbb{R}^{T \times d}$ The *input embedding matrix* at a given layer. Row t is the representation of the t-th token, denoted $h_t \in \mathbb{R}^d$.

$H_s \in \mathbb{R}^{\mathcal{P}_\ell \times d}$ The *attention segment* of H: the sub-matrix formed by the first $\mathcal{P}_\ell$ rows, $H_s = H[0 : \mathcal{P}_\ell, :]$.

$H_l \in \mathbb{R}^{(T-\mathcal{P}_\ell) \times d}$ The *SSM segment* of H: the sub-matrix formed by the remaining rows, $H_l = H[\mathcal{P}_\ell : T, :]$.

$h_k^{\text{ssm}} \in \mathbb{R}^{d \times N}$ The *SSM hidden state* at position k within the SSM segment, where k is a local index $k \in \{0, \ldots, T - \mathcal{P}_\ell - 1\}$.

$y_s \in \mathbb{R}^{\mathcal{P}_\ell \times d}$ The output of the attention mechanism applied to the attention segment.

$y_l \in \mathbb{R}^{(T-\mathcal{P}_\ell) \times d}$ The output of the SSM applied to the SSM segment.

$y \in \mathbb{R}^{T \times d}$ The full layer output, $y = [y_s; y_l]$ (row-wise concatenation).

2.3 TransPoint Scheduling

$\mathcal{P}_\ell \in \{0, \ldots, T\}$ The *TransPoint* at layer ℓ: the token position at which the model switches from Transformer attention to SSM. When $\mathcal{P}_\ell = 0$ the layer is purely SSM; when $\mathcal{P}_\ell = T$ it is purely Transformer.

$, \ldots, \mathcal{P}_L) \in \{0, \ldots, T\}^L$ The *TransPoint schedule*: the vector of all TransPoints across layers. The optimal schedule found by Li et al. [2026] cycles every 8 layers through $\{0, 128, 256, 512, 1024, 2048, 4096, 8192\}$.

$\tau_\ell(t) \in \{0, 1\}$ A binary *mechanism indicator*, $\tau_\ell(t) = \mathbf{1}[t \geq \mathcal{P}_\ell]$, equal to 0 (attention mode) when $t < \mathcal{P}_\ell$ and 1 (SSM mode) when $t \geq \mathcal{P}_\ell$.

2.4 Shared Weight Matrix and Its Dual-Form Role

$\mathbf{W} \in \mathbb{R}^{d \times d}$ The single *shared weight matrix* of TransMamba, which plays the role of $\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$ in the attention mechanism and simultaneously $\mathbf{W}_C$, $\mathbf{W}_B$, $\mathbf{W}_x$ in the SSM (Table 1 of Li et al. 2026). The identification is $Q \leftrightarrow C$, $K \leftrightarrow B$, $V \leftrightarrow x$.

$\mathbf{W}_\Delta \in \mathbb{R}^{d \times d}$ The projection matrix that computes the SSM step-size Δ.

$\mathbf{W}_A \in \mathbb{R}^{d \times N}$ The log-parameterised SSM transition matrix weight, so that $A = \exp(-\Delta \cdot e^{\log \mathbf{W}_A})$.

$\mathbf{W}_Z \in \mathbb{R}^{d \times d}$ The SSM gating projection.

$\delta : \mathbb{R} \to \mathbb{R}$ The *nonlinear activation function*, element-wise SiLU: $\delta(x) = x \cdot \sigma(x)$, $\sigma(x) = (1 + e^{-x})^{-1}$.

2.5 Attention Mechanism Tensors

$Q \in \mathbb{R}^{\mathcal{P}_\ell \times d}$ The *query matrix*, $Q = \delta(H_s \mathbf{W}_Q)$.

$K \in \mathbb{R}^{\mathcal{P}_\ell \times d}$ The *key matrix*, $K = \delta(H_s \mathbf{W}_K)$.

$V \in \mathbb{R}^{\mathcal{P}_\ell \times d}$ The *value matrix*, $V = \delta(H_s \mathbf{W}_V)$.

$A^{\times} \in \mathbb{R}^{\mathcal{P}_\ell \times \mathcal{P}_\ell}$ The *lower-triangular cumulative-product matrix* of the SSM transition scalars:

$$(A^{\times})_{ij} = \begin{cases} \prod_{k=j}^{i} A_k & i \geq j, \\ 0 & i < j, \end{cases} \tag{1}$$

where $A_k \in \mathbb{R}$ is the scalar transition at position k. Explicitly (Appendix A.2.1 of Li et al. 2026), $A^{\times}$ is lower-triangular with $(A^{\times})_{ij} = \prod_{k=j}^{i} A_k$ for $i \geq j$.

$L \in \{0,1\}^{\mathcal{P}_\ell \times \mathcal{P}_\ell}$ The *causal mask*: $L_{ij} = 1$ if $i \geq j$, else 0.

2.6 SSM Mechanism Tensors

$C \in \mathbb{R}^{(T-\mathcal{P}_\ell)\times d}$ The *SSM output projection matrix*, $C = \delta(H_l \mathbf{W}_C)$. Under the duality, $C \equiv Q$ computed from H_l.

$B \in \mathbb{R}^{(T-\mathcal{P}_\ell)\times d}$ The *SSM input projection matrix*, $B = \delta(H_l \mathbf{W}_B)$. Under the duality, $B \equiv K$ computed from H_l.

$x \in \mathbb{R}^{(T-\mathcal{P}_\ell)\times d}$ The *SSM input signal*, $x = \delta(H_l \mathbf{W}_x) \circ \Delta$. Under the duality, $x \equiv V$ computed from H_l.

$\Delta \in \mathbb{R}^{(T-\mathcal{P}_\ell)\times d}$ The *step-size matrix* (element-wise positive), $\Delta = \sigma(H_l \mathbf{W}_\Delta + b_\Delta)$, with bias $b_\Delta \in \mathbb{R}^d$.

$A \in \mathbb{R}^{(T-\mathcal{P}_\ell)\times d}$ The *discretised transition matrix (scalars)*, $A = \exp(-\Delta \circ e^{\log \mathbf{W}_A})$, where $\circ$ denotes element-wise (Hadamard) product.

$Z \in \mathbb{R}^{(T-\mathcal{P}_\ell)\times d}$ The *SSM gating signal*, $Z = \delta(H_l \mathbf{W}_Z)$.

$h_k^{\text{ssm}} \in \mathbb{R}^{d_h \times N_{\text{st}}}$ The *SSM hidden state* at local position k, satisfying $h_k^{\text{ssm}} = A_{k-1} \odot h_{k-1}^{\text{ssm}} + B_k \circ \Delta_k \circ x_k$, where $\odot$ denotes row-wise broadcasting of the scalar A_{k-1} across the state matrix dimensions.

2.7 Mixture-of-Experts Notation

$N_e \in \mathbb{N}$ The *number of expert groups*. In the DeepSeekMoE framework [Dai et al., 2024], this is the count of distinct expert modules before fine-grained splitting.

$m \in \mathbb{N}$ The *fine-grained splitting factor*. DeepSeekMoE splits each expert's intermediate dimension into m smaller experts, so the total pool has mN_e experts each with intermediate dimension $d_m = d_{\text{ffn}}/m$.

$N_s \in \mathbb{N}$ The *number of shared experts*: experts that are *always activated* for every token, regardless of routing.

$N_r \in \mathbb{N}$ The *number of routed experts*, $N_r = mN_e - N_s$.

$K \in \mathbb{N}$ The *number of routed experts activated per token*, $1 \leq K \leq N_r$. The total active experts per token is $N_s + K$.

$\mathbf{W}^{(i)} \in \mathbb{R}^{d\times d}$ The weight matrix of the i-th expert in the W-MoE pool, $i \in \{1, \ldots, mN_e\}$. Experts $i \in \{1, \ldots, N_s\}$ are shared; experts $i \in \{N_s + 1, \ldots, mN_e\}$ are routed.

$g_\phi : \mathbb{R}^d \to \mathbb{R}^{mN_e}$ The *router function* parameterised by ϕ. Given a single token representation $h_t \in \mathbb{R}^d$, it outputs a logit vector:

$$\boldsymbol{\alpha}_t = g_\phi(h_t) = h_t \mathbf{W}_\phi \in \mathbb{R}^{mN_e}, \tag{2}$$

where $\mathbf{W}_\phi \in \mathbb{R}^{d\times mN_e}$ is the router weight matrix.

$s_{t,i} \in [0,1]$ The *routing score* of expert i for token t, i.e. the softmax of the router logits: $s_{t,i} = e^{\boldsymbol{\alpha}_{t,i}} / \sum_{j=1}^{mN_e} e^{\boldsymbol{\alpha}_{t,j}}$.

$\mathcal{E}_t \subseteq \{N_s + 1, \ldots, mN_e\}$ The *activated routed expert set* for token t, i.e. the indices of the K routed experts with the largest scores: $\mathcal{E}_t = \text{top-k}(\{s_{t,i}\}_{i=N_s+1}^{mN_e},\ K)$.

$\hat{s}_{t,i} \in [0,1]$ The *normalised routing weight* for activated expert i and token t: $\hat{s}_{t,i} = s_{t,i} / \sum_{j\in\mathcal{E}_t} s_{t,j}$ if $i \in \mathcal{E}_t$, else 0.

$\mathbf{W}_t^* \in \mathbb{R}^{d\times d}$ The *effective weight matrix* for token t, combining shared and routed experts with score weighting (formally: $\mathbf{W}_t^* = \frac{1}{N_s}\sum_{i=1}^{N_s} \mathbf{W}^{(i)} + \sum_{i\in\mathcal{E}_t} \hat{s}_{t,i}\mathbf{W}^{(i)}$).

$f_i \in [0,1]$ The *expert load fraction*: fraction of batch tokens routed to expert i, $f_i = T_{\text{batch}}^{-1} \sum_{t=1}^{T_{\text{batch}}} \mathbf{1}[i \in \mathcal{E}_t]$.

$P_i \in [0,1]$ The *mean routing probability* to expert i: $P_i = T_{\text{batch}}^{-1} \sum_{t=1}^{T_{\text{batch}}} s_{t,i}$.

2.8 Loss Function Components

$\mathcal{L}_{\text{CE}}$ The standard next-token-prediction *cross-entropy loss* over the vocabulary.

$\mathcal{L}_{\text{bal}}^{(a)}$ The *attention-segment load-balancing loss* (computed only over tokens $t < \mathcal{P}_\ell$; derived in Section 8).

$\mathcal{L}_{\text{bal}}^{(s)}$ The *SSM-segment load-balancing loss* (computed only over tokens $t \geq \mathcal{P}_\ell$; derived in Section 8).

$\mathcal{L}_{\text{total}}$ The *total training loss*: $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda_a \mathcal{L}_{\text{bal}}^{(a)} + \lambda_s \mathcal{L}_{\text{bal}}^{(s)}$, with $\lambda_a, \lambda_s > 0$ mechanism-specific balancing coefficients (see Section 8).

3 Background: TransMamba and DeepSeekMoE

3.1 TransMamba: The Dual-Form Architecture

We restate the forward pass of a single TransMamba layer [Li et al., 2026] in the notation of Section 2.

Step 1: Partition the sequence. Given $H \in \mathbb{R}^{T\times d}$ and TransPoint $\mathcal{P}_\ell$, form:

$$H_s = H[0 : \mathcal{P}_\ell,\ :], \qquad dH_l = H[\mathcal{P}_\ell : T,\ :]. \tag{3}$$

Step 2: Shared projections. (Eqs. (1) and (3) of Li et al. 2026) Both segments are projected by the same matrix $\mathbf{W}$:

$$Q = \delta(H_s \mathbf{W}_Q), \qquad K = \delta(H_s \mathbf{W}_K), \qquad V = \delta(H_s \mathbf{W}_V), \tag{4}$$
$$C = \delta(H_l \mathbf{W}_C), \qquad B = \delta(H_l \mathbf{W}_B), \qquad x = \delta(H_l \mathbf{W}_x), \tag{5}$$

with $\mathbf{W}_Q \equiv \mathbf{W}_C$, $\mathbf{W}_K \equiv \mathbf{W}_B$, $\mathbf{W}_V \equiv \mathbf{W}_x$ (all equal to $\mathbf{W}$).

Step 3: Attention output. (Eq. (2) of Li et al. 2026)

$$y_s = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V \in \mathbb{R}^{\mathcal{P}_\ell \times d}. \tag{6}$$

Step 4: SSM parameters. (Eq. (3) of Li et al. 2026)

$$\Delta = \sigma(H_l \mathbf{W}_\Delta + b_\Delta) \in \mathbb{R}^{(T-\mathcal{P}_\ell)\times d}, \tag{7}$$
$$A = \exp(-\Delta \circ e^{\log \mathbf{W}_A}) \in \mathbb{R}^{(T-\mathcal{P}_\ell)\times d}. \tag{8}$$

Step 5: Memory Converter. (Eqs. (7)–(9) of Li et al. 2026) The initial SSM hidden state is derived from the attention key and value matrices without additional parameters:

$$h_s = (A^\times \circ K^\top) V \in \mathbb{R}^{\mathcal{P}_\ell \times d \times N}, \qquad dh_0^{\mathrm{ssm}} = h_s[-1] \in \mathbb{R}^{d \times N}. \tag{9}$$

Step 6: SSM recurrence. (Eq. (4) of Li et al. 2026)

$$h_k^{\mathrm{ssm}} = A_{k-1} \odot h_{k-1}^{\mathrm{ssm}} + B_k \circ \Delta_k \circ x_k, \qquad dy_k = C_k h_k^{\mathrm{ssm}}, \tag{10}$$

and $y_l = [y_0, y_1, \ldots] \in \mathbb{R}^{(T-\mathcal{P}_\ell)\times d}$.

Step 7: Concatenate. (Eq. (6) of Li et al. 2026)

$$y = [y_s;\, y_l] \in \mathbb{R}^{T \times d}. \tag{11}$$

The per-layer FLOPs are (Table 2 of Li et al. 2026):

$$\mathrm{FLOPs}_{\mathrm{TM}}^{(\ell)} = \mathcal{O}\big(\mathcal{P}_\ell^2 N + (T - \mathcal{P}_\ell) N^2\big). \tag{12}$$

3.2 DeepSeekMoE: Fine-Grained Shared-Expert Isolation

We restate the DeepSeekMoE feed-forward computation [Dai et al., 2024] in our notation, as this is the MoE variant we incorporate.

Let $u \in \mathbb{R}^d$ be a single token representation entering the MoE FFN. DeepSeekMoE introduces two modifications over sparse MoE:

(a) Fine-grained expert segmentation. Instead of N_e experts each with intermediate dimension d_{ffn}, the intermediate dimension is split into mN_e experts each with intermediate dimension $d_m = d_{\mathrm{ffn}}/m$. This increases the total expert count while keeping each expert small and specialised.

(b) Shared expert isolation. N_s experts (with full intermediate dimension d_{ffn}) are designated as *shared*: they are activated for every token. The remaining $N_r = mN_e - N_s$ experts are *routed*.

The output of the DeepSeekMoE FFN for token u is:

$$\text{FFN}_{\text{MoE}}(u) = \underbrace{\sum_{i=1}^{N_s} \text{FFN}^{(i)}(u)}_{\text{shared experts}} + \underbrace{\sum_{i \in \mathcal{E}_t} \hat{s}_{t,i} \, \text{FFN}^{(i)}(u)}_{\text{routed experts}}, \tag{13}$$

where $\text{FFN}^{(i)}(u) = \mathbf{W}_{\text{up}}^{(i)}(\delta(\mathbf{W}_{\text{gate}}^{(i)} u))$ is the i-th expert's two-matrix feed-forward computation, and $\mathcal{E}_t, \hat{s}_{t,i}$ are as defined in (**??**) and (**??**).

The load-balancing auxiliary loss of Dai et al. [2024] is:

$$\mathcal{L}_{\text{bal}}^{\text{DSK}} = \alpha \sum_{i=1}^{N_r} f_i P_i, \tag{14}$$

where $\alpha > 0$ is a coefficient, f_i is the expert load fraction (**??**), and P_i is the mean routing probability (**??**).

4 The W-MoE: Expert Pool on the Shared Weight Matrix

4.1 Motivation and the Coupling Constraint

In standard TransMamba, a single $\mathbf{W} \in \mathbb{R}^{d \times d}$ drives both the attention projections (4) and the SSM projections (5). The W-MoE replaces this single matrix with a pool of experts $\{\mathbf{W}^{(i)}\}_{i=1}^{mN_e}$ and selects a token-specific effective matrix $\mathbf{W}_t^*$ via routing.

The fundamental constraint is the *coupling constraint*: the same expert selection $\mathcal{E}_t$ and the same effective matrix $\mathbf{W}_t^*$ must be used for both the attention branch (producing Q, K, V from H_s) and the SSM branch (producing C, B, x from H_l) for any given token t. This is because the algebraic duality $Q \leftrightarrow C$, $K \leftrightarrow B$, $V \leftrightarrow x$ that TransMamba relies upon requires both mechanisms to use the same linear transformation. Violating this constraint by using different experts for the two branches would break the Memory Converter (Eq. (9)), whose derivation assumes that K and V in the attention segment and C, B, x in the SSM segment are related by the same $\mathbf{W}$.

Definition 4.1 (W-MoE Expert Pool)**.** The W-MoE expert pool is a collection $\{\mathbf{W}^{(i)}\}_{i=1}^{mN_e}$ of matrices in $\mathbb{R}^{d \times d}$, partitioned into:

- *Shared experts*: $\{\mathbf{W}^{(i)}\}_{i=1}^{N_s}$, always activated.
- *Routed experts*: $\{\mathbf{W}^{(i)}\}_{i=N_s+1}^{mN_e}$, selectively activated by the router.

The effective weight matrix for token t is $\mathbf{W}_t^*$ as defined in (**??**), and it is applied identically to both the attention segment H_s and the SSM segment H_l.

4.2 W-MoE Router

The router operates at the token level. For a token at position t with representation $h_t \in \mathbb{R}^d$ (the t-th row of H), the router computes:

$$\boldsymbol{\alpha}_t = h_t \mathbf{W}_\phi \in \mathbb{R}^{mN_e}, \tag{15}$$

$$s_{t,i} = \text{softmax}(\boldsymbol{\alpha}_t)_i, \quad i = 1, \ldots, mN_e, \tag{16}$$

$$\mathcal{E}_t = \text{top-k}\big(\{s_{t,i}\}_{i=N_s+1}^{mN_e},\, K\big). \tag{17}$$

The effective weight matrix is then:

$$\mathbf{W}_t^* = \frac{1}{N_s} \sum_{i=1}^{N_s} \mathbf{W}^{(i)} + \sum_{i \in \mathcal{E}_t} \hat{s}_{t,i}\, \mathbf{W}^{(i)}, \tag{18}$$

where $\hat{s}_{t,i}$ normalises over activated routed experts as in (**??**).

4.3 W-MoE Forward Pass

With $\mathbf{W}_t^*$ defined per-token, the attention projections for the attention segment become:

$$Q_t = \delta(h_t^s \mathbf{W}_t^*), \quad K_t = \delta(h_t^s \mathbf{W}_t^*), \quad V_t = \delta(h_t^s \mathbf{W}_t^*), \quad t < \mathcal{P}_\ell, \tag{19}$$

where $h_t^s \in \mathbb{R}^d$ is the t-th row of H_s. The three projections produce the same numerical output because they share $\mathbf{W}_t^*$; the separate role of each (as Q, K, or V) is determined by how the output is used downstream, consistent with the TransMamba design.

For the SSM segment:

$$C_t = \delta(h_t^l \mathbf{W}_t^*), \quad B_t = \delta(h_t^l \mathbf{W}_t^*), \quad x_t = \delta(h_t^l \mathbf{W}_t^*), \quad t \geq \mathcal{P}_\ell, \tag{20}$$

where h_t^l is the $(t - \mathcal{P}_\ell)$-th row of H_l.

Remark 4.2 (Computational cost of W-MoE)**.** For each token t, computing $\mathbf{W}_t^*$ requires $N_s + K$ matrix lookups and a weighted sum over d^2 parameters. The effective computation for one token is:

$$\text{FLOPs}_t^{\mathbf{W}^*} = (N_s + K) \cdot d^2 + (K - 1) \cdot d^2 = (2K + N_s - 1)d^2. \tag{21}$$

Over the full sequence of T tokens: $\text{FLOPs}_{\text{seq}}^{\text{W-MoE}} = T(2K + N_s - 1)d^2$.

5 The SSM-MoE: Expert Pool on SSM Projection Layers

5.1 Motivation

The SSM segment of TransMamba uses three additional projection matrices $\mathbf{W}_\Delta$, $\mathbf{W}_A$, and $\mathbf{W}_Z$ that have no equivalent in the attention segment. These matrices compute the step-size Δ, the transition A, and the gating signal Z, all of which are *data-dependent* in Mamba [Gu and Dao, 2023] and control the selective memory mechanism. Making these into a mixture of experts allows the SSM to specialise different experts for different types of long-range dependencies, a capability absent from single-expert SSMs.

We follow the *shared routing* principle of Routing Mamba [Katharopoulos et al., 2025]: the same routing decision $\mathcal{E}_t^{\text{ssm}}$ is shared between the Δ and x projections (since they are directly multiplied in the recurrence $h_k = A_{k-1}h_{k-1} + B_k \Delta_k x_k$, making them functionally co-dependent).

5.2 SSM-MoE Expert Pools

We define three separate expert pools, one per SSM projection:

Definition 5.1 (SSM-MoE Expert Pools)**.** For the SSM segment, define pools:

$$\mathcal{F}_\Delta = \{\mathbf{W}_\Delta^{(i)}\}_{i=1}^{mN_e^{\text{ssm}}}, \quad \mathbf{W}_\Delta^{(i)} \in \mathbb{R}^{d\times d}, \tag{22}$$

$$\mathcal{F}_A = \{\mathbf{W}_A^{(i)}\}_{i=1}^{mN_e^{\text{ssm}}}, \quad \mathbf{W}_A^{(i)} \in \mathbb{R}^{d\times N}, \tag{23}$$

$$\mathcal{F}_Z = \{\mathbf{W}_Z^{(i)}\}_{i=1}^{mN_e^{\text{ssm}}}, \quad \mathbf{W}_Z^{(i)} \in \mathbb{R}^{d\times d}, \tag{24}$$

each with N_s^{ssm} shared and N_r^{ssm} routed experts.

5.3 Shared SSM Router

A single router $g_\phi^{\text{ssm}} : \mathbb{R}^d \to \mathbb{R}^{mN_e^{\text{ssm}}}$ is shared across all three SSM projection pools. For a token at SSM-segment position k with representation $h_k^l \in \mathbb{R}^d$ (the k-th row of H_l):

$$\boldsymbol{\alpha}_k^{\text{ssm}} = h_k^l \mathbf{W}_\phi^{\text{ssm}} \in \mathbb{R}^{mN_e^{\text{ssm}}}, \qquad d\mathcal{E}_k^{\text{ssm}} = \text{top-k}\big(\{s_{k,i}^{\text{ssm}}\}_{i=N_s^{\text{ssm}}+1}^{mN_e^{\text{ssm}}}, K^{\text{ssm}}\big). \tag{25}$$

The effective SSM projection matrices for position k are:

$$\mathbf{W}_\Delta^{*,k} = \frac{1}{N_s^{\text{ssm}}} \sum_{i=1}^{N_s^{\text{ssm}}} \mathbf{W}_\Delta^{(i)} + \sum_{i\in\mathcal{E}_k^{\text{ssm}}} \hat{s}_{k,i}^{\text{ssm}} \mathbf{W}_\Delta^{(i)}, \tag{26}$$

$$\mathbf{W}_A^{*,k} = \frac{1}{N_s^{\text{ssm}}} \sum_{i=1}^{N_s^{\text{ssm}}} \mathbf{W}_A^{(i)} + \sum_{i\in\mathcal{E}_k^{\text{ssm}}} \hat{s}_{k,i}^{\text{ssm}} \mathbf{W}_A^{(i)}, \tag{27}$$

$$\mathbf{W}_Z^{*,k} = \frac{1}{N_s^{\text{ssm}}} \sum_{i=1}^{N_s^{\text{ssm}}} \mathbf{W}_Z^{(i)} + \sum_{i\in\mathcal{E}_k^{\text{ssm}}} \hat{s}_{k,i}^{\text{ssm}} \mathbf{W}_Z^{(i)}. \tag{28}$$

5.4 SSM-MoE Forward Pass

With position-dependent effective projections, the SSM parameters become:

$$\Delta_k = \sigma(h_k^l \mathbf{W}_\Delta^{*,k} + b_\Delta), \tag{29}$$

$$A_k = \exp(-\Delta_k \circ e^{\log \mathbf{W}_A^{*,k}}), \tag{30}$$

$$Z_k = \delta(h_k^l \mathbf{W}_Z^{*,k}). \tag{31}$$

The recurrence (10) then uses these position-specific parameters, with C_k, B_k, x_k coming from the W-MoE forward pass (20).

6 The FFN-MoE: Expert Pool on the Feed-Forward Sublayer

6.1 Architecture

After the attention+SSM block and the residual connection, each TransMamba layer has a standard "Norm & Feed Forward" sublayer (Figure 1 of Li et al. 2026). We replace this with a DeepSeekMoE FFN exactly as in (13).

Let $u_t \in \mathbb{R}^d$ be the RMSNorm-normalised residual of token t entering the FFN sublayer. The FFN-MoE output is:

$$\text{FFN-MoE}(u_t) = \sum_{i=1}^{N_s^{\text{ffn}}} \text{FFN}^{(i)}(u_t) + \sum_{i \in \mathcal{E}_t^{\text{ffn}}} \hat{s}_{t,i}^{\text{ffn}} \, \text{FFN}^{(i)}(u_t), \tag{32}$$

where each expert $\text{FFN}^{(i)}$ has the two-matrix gated structure:

$$\text{FFN}^{(i)}(u) = \mathbf{W}_{\text{down}}^{(i)} \big(\delta(\mathbf{W}_{\text{gate}}^{(i)} u) \circ (\mathbf{W}_{\text{up}}^{(i)} u) \big), \tag{33}$$

with $\mathbf{W}_{\text{gate}}^{(i)}, \mathbf{W}_{\text{up}}^{(i)} \in \mathbb{R}^{d_m \times d}$ and $\mathbf{W}_{\text{down}}^{(i)} \in \mathbb{R}^{d \times d_m}$. The FFN router g_ϕ^{ffn} and the top-K^{ffn} selection follow exactly (15)–(17) with the superscript "ffn".

7 Design Problem 1: The Expert-Conditioned Memory Converter

7.1 Statement of the Problem

Design Problem 7.1 (Expert-Conditioned Memory Converter)**.** In the original TransMamba Memory Converter (Eq. (9)), the matrices K and V (produced from H_s by the single matrix $\mathbf{W}$) and the matrices C, B, x (produced from H_l by the same $\mathbf{W}$) are algebraically consistent: they arise from the same linear transformation. In TransMamba-MoE, different tokens in H_s may use different effective matrices $\{\mathbf{W}_t^*\}_{t=0}^{\mathcal{P}_\ell - 1}$. The Memory Converter (9) uses K and V from the last token of the attention segment to initialise h_0^{ssm}:

$$h_0^{\text{ssm}} = (A^\times \circ K^\top) V \big|_{[-1]}. \tag{34}$$

The question is: for which value of $\mathbf{W}^*$ is this initialisation valid when different tokens used different $\mathbf{W}_t^*$? Specifically, what is the correct expert-conditioned initial state h_0^{ssm} that maintains lossless information transfer from the attention segment to the SSM segment?

7.2 Derivation of the Expert-Conditioned Memory Converter

We begin from the full SSM hidden-state matrix form (Eq. (8) of Li et al. 2026):

$$H^{\text{ssm}} = (A^\times \circ B^\top)(\Delta \circ x), \tag{35}$$

where $H^{\text{ssm}} \in \mathbb{R}^{\mathcal{P}_\ell \times d \times N}$ is the stacked hidden-state matrix over the attention segment. By the algebraic duality (Table 1 of Li et al. 2026), with $B \equiv K$ and $x \equiv V$:

$$H^{\text{ssm}} \approx (A^\times \circ K^\top) V. \tag{36}$$

In TransMamba-MoE, the t-th rows of K and V are:

$$K_t = \delta(h_t^s \mathbf{W}_t^*), \qquad dV_t = \delta(h_t^s \mathbf{W}_t^*), \tag{37}$$

so the product $K^\top V$ becomes a sum of rank-one outer products, each weighted by the corresponding routing scores:

$$K^\top V = \sum_{t=0}^{\mathcal{P}_\ell - 1} K_t^\top V_t = \sum_{t=0}^{\mathcal{P}_\ell - 1} \left[\delta(h_t^s \mathbf{W}_t^*) \right]^\top \left[\delta(h_t^s \mathbf{W}_t^*) \right]. \tag{38}$$

We decompose $\mathbf{W}_t^*$ as in (18) and define a *token-specific residual term* R_t:

$$\mathbf{W}_t^* = \bar{\mathbf{W}} + R_t, \tag{39}$$

where $\bar{\mathbf{W}} = \frac{1}{N_s}\sum_{i=1}^{N_s} \mathbf{W}^{(i)}$ is the mean shared-expert matrix (constant across all tokens), and $R_t = \sum_{i\in\mathcal{E}_t} \hat{s}_{t,i}\mathbf{W}^{(i)}$ is the token-specific routed contribution.

By a first-order Taylor expansion of δ around the shared baseline $h_t^s\bar{\mathbf{W}}$:

$$\delta(h_t^s\mathbf{W}_t^*) \approx \delta(h_t^s\bar{\mathbf{W}}) + \delta'(h_t^s\bar{\mathbf{W}}) \circ (h_t^s R_t), \tag{40}$$

where $\delta'(x) = \sigma(x)(1 + x(1 - \sigma(x)))$ is the derivative of SiLU, applied element-wise. Substituting (40) into (38) and retaining only first-order terms in R_t:

$$\begin{aligned} K_t^\top V_t &\approx \left[\bar{K}_t + \delta'(\bar{K}_t) \circ (h_t^s R_t)\right]^\top \left[\bar{V}_t + \delta'(\bar{V}_t) \circ (h_t^s R_t)\right] \\ &\approx \bar{K}_t^\top \bar{V}_t + \bar{K}_t^\top[\delta'(\bar{V}_t) \circ (h_t^s R_t)] + [\delta'(\bar{K}_t) \circ (h_t^s R_t)]^\top \bar{V}_t, \end{aligned} \tag{41}$$

where $\bar{K}_t = \delta(h_t^s\bar{\mathbf{W}})$, $\bar{V}_t = \delta(h_t^s\bar{\mathbf{W}})$.

Definition 7.2 (Expert-Conditioned Memory Converter)**.** The *Expert-Conditioned Memory Converter* (MCE) computes the initial SSM hidden state as:

$$\boxed{h_0^{\text{ssm}} = \left(A^\times \circ \bar{K}^\top\right)\bar{V}\big|_{[-1]} + \sum_{t=0}^{\mathcal{P}_\ell - 1} (A^\times)_{\mathcal{P}_\ell - 1, t} \cdot \Phi_t(K_t, V_t, \bar{K}_t, \bar{V}_t, R_t),} \tag{42}$$

where $\bar{K}, \bar{V}$ are the key and value matrices computed using the mean shared matrix $\bar{\mathbf{W}}$, and the correction term Φ_t is:

$$\Phi_t(K_t, V_t, \bar{K}_t, \bar{V}_t, R_t) = \bar{K}_t^\top\left[\delta'(\bar{V}_t) \circ (h_t^s R_t)\right] + \left[\delta'(\bar{K}_t) \circ (h_t^s R_t)\right]^\top \bar{V}_t. \tag{43}$$

This requires no additional learned parameters beyond those already present in the W-MoE pool.

Theorem 7.3 (Correctness of MCE)**.** *Let h_0^* be the true initial SSM hidden state computed from the exact effective matrices $\{\mathbf{W}_t^*\}_{t=0}^{\mathcal{P}_\ell - 1}$, and let h_0^{MCE} be the approximation given by (42). Then:*

$$\left\|h_0^* - h_0^{\text{MCE}}\right\|_F \le C_\delta \sum_{t=0}^{\mathcal{P}_\ell - 1} (A^\times)^2_{\mathcal{P}_\ell - 1, t} \left\|R_t\right\|_F^2 \left\|h_t^s\right\|_2^2, \tag{44}$$

where $C_\delta = \sup_x |\delta''(x)|/2$ is the half-norm of the second derivative of SiLU (which is bounded: $\delta'' \in L^\infty(\mathbb{R})$ since SiLU is smooth).

Proof. The error arises from omitting the second-order term in the Taylor expansion (40):

$$\delta(h_t^s\mathbf{W}_t^*) - \delta(h_t^s\bar{\mathbf{W}}) - \delta'(h_t^s\bar{\mathbf{W}}) \circ (h_t^s R_t) = \frac{1}{2}\delta''(\xi_t) \circ (h_t^s R_t)^2 \tag{45}$$

for some ξ_t between $h_t^s\bar{\mathbf{W}}$ and $h_t^s\mathbf{W}_t^*$ (by the Lagrange form of the remainder). Taking Frobenius norms and using $|\delta''| \le C_\delta$ everywhere gives the bound (44). The error vanishes as $\|R_t\|_F \to 0$, i.e. as expert specialisation diminishes towards a single shared matrix. □

Remark 7.4 (When is the approximation exact?)**.** If all tokens in the attention segment share the same effective matrix (i.e. $R_t = R$ constant, or equivalently a single-expert model), then Φ_t is also constant and the correction reduces to a global shift, recovering the exact original Memory Converter (9). The approximation is therefore consistent with the original TransMamba in the degenerate single-expert case.

Remark 7.5 (Exact computation of the correction)**.** In practice, computing Φ_t exactly requires materialising R_t for each attention-segment token, which is already available from the W-MoE routing computation. The overhead is one element-wise product and two matrix additions per attention-segment token: $\mathcal{O}(\mathcal{P}_\ell d^2)$ FLOPs total, which is the same order as the attention computation itself. The MCE can therefore be computed exactly (to numerical precision) without approximation, and Theorem 7.3 shows the Taylor-based representation is an approximation only for analytical purposes.

8 Design Problem 2: Mechanism-Aware Load Balancing

8.1 Statement of the Problem

Design Problem 8.1 (Mechanism-Aware Load Balancing)**.** In standard MoE, the load-balancing loss (14) is applied globally across all tokens. In TransMamba-MoE, a sequence of T tokens is partitioned at $\mathcal{P}_\ell$ into an attention segment ($\mathcal{P}_\ell$ tokens) and an SSM segment ($T - \mathcal{P}_\ell$ tokens). Tokens in different segments have structurally different representations (attention-processed vs. SSM-processed), so expert preference distributions differ systematically between segments. A single global load-balancing loss will either over-balance (suppressing the natural segment-level specialisation) or under-balance (allowing one segment to collapse routing to a small expert subset). How should the load-balancing loss be correctly decomposed across mechanisms?

8.2 Mechanism-Aware Load Balancing: Formulation

Definition 8.2 (Segment-specific load statistics)**.** For the W-MoE, define separate load statistics for each mechanism:

$$f_i^{(a)} = \frac{1}{\mathcal{P}_\ell} \sum_{t=0}^{\mathcal{P}_\ell - 1} \mathbf{1}[i \in \mathcal{E}_t], \qquad dP_i^{(a)} = \frac{1}{\mathcal{P}_\ell} \sum_{t=0}^{\mathcal{P}_\ell - 1} s_{t,i}, \tag{46}$$

$$f_i^{(s)} = \frac{1}{T - \mathcal{P}_\ell} \sum_{t=\mathcal{P}_\ell}^{T-1} \mathbf{1}[i \in \mathcal{E}_t], \qquad dP_i^{(s)} = \frac{1}{T - \mathcal{P}_\ell} \sum_{t=\mathcal{P}_\ell}^{T-1} s_{t,i}. \tag{47}$$

Definition 8.3 (Mechanism-aware load balancing loss)**.** The mechanism-aware load-balancing losses for the W-MoE are:

$$\mathcal{L}_{\text{bal}}^{(a)} = \alpha_a \sum_{i=N_s+1}^{mN_e} f_i^{(a)} P_i^{(a)}, \tag{48}$$

$$\mathcal{L}_{\text{bal}}^{(s)} = \alpha_s \sum_{i=N_s+1}^{mN_e} f_i^{(s)} P_i^{(s)}, \tag{49}$$

with *mechanism-specific* coefficients $\alpha_a, \alpha_s > 0$ to be set separately. For the SSM-MoE, an analogous loss $\mathcal{L}_{\text{bal}}^{(\text{ssm-proj})}$ is defined over the SSM-segment tokens only. The FFN-MoE uses a standard global balancing loss $\mathcal{L}_{\text{bal}}^{(\text{ffn})}$.

Theorem 8.4 (Optimality of segment-wise balancing)**.** *Let $f_i^{(a)}$, $f_i^{(s)}$ denote the segment-specific load fractions as in (46)–(47). Under the idealised assumption that the optimal expert routing distributions for the attention and SSM segments are independent, the minimiser of the total loss $\mathcal{L}_{\text{total}}$ (**??**) with segment-specific coefficients $\alpha_a \neq \alpha_s$ is not achievable by any global coefficient α_{global} that minimises a single loss $\alpha_{\text{global}} \sum_i f_i P_i$.*

Proof. The global loss conflates two objectives: balance within the attention segment and balance within the SSM segment. Denote the attention-segment optimal routing distribution by $\{p_i^{(a)*}\}$ and the SSM-segment optimal by $\{p_i^{(s)*}\}$. In general $p_i^{(a)*} \neq p_i^{(s)*}$ for some i, since attention and SSM representations lie in different subspaces of $\mathbb{R}^d$ (they are produced by different mechanisms). The gradient of the global loss at the joint minimiser is:

$$\nabla_\phi \alpha_{\text{global}} \sum_i f_i P_i = \alpha_{\text{global}} \nabla_\phi \Big[\frac{\mathcal{P}_\ell}{T} \sum_i f_i^{(a)} P_i^{(a)} + \frac{T - \mathcal{P}_\ell}{T} \sum_i f_i^{(s)} P_i^{(s)} \Big]. \tag{50}$$

This is a fixed linear combination with weights $\mathcal{P}_\ell / T$ and $(T - \mathcal{P}_\ell)/T$, determined by the TransPoint rather than by the semantic difficulty of balancing in each segment. The segment-specific formulation allows α_a and α_s to be set independently (e.g. by hyperparameter search or theoretical analysis), achieving lower total loss for any joint distribution where $p_i^{(a)*} \neq p_i^{(s)*}$, which holds generically. □

8.3 Setting α_a and α_s

A practical heuristic for initialising α_a and α_s is derived from the relative segment lengths and the TransPoint schedule. Define the *balance difficulty ratio*:

$$\rho_\ell = \frac{\mathcal{P}_\ell}{T - \mathcal{P}_\ell} \tag{51}$$

as the ratio of attention-to-SSM token counts at layer ℓ. When $\rho_\ell \ll 1$ (SSM segment dominates), the attention-segment router sees fewer training signals and requires stronger regularisation; conversely for $\rho_\ell \gg 1$. We propose:

$$\alpha_a = \alpha_0 \cdot \frac{1}{\rho_\ell}, \qquad d\alpha_s = \alpha_0 \cdot \rho_\ell, \tag{52}$$

where $\alpha_0 > 0$ is a single shared hyperparameter. This ensures that the product $\alpha_a \cdot \mathcal{P}_\ell = \alpha_s \cdot (T - \mathcal{P}_\ell) = \alpha_0 T$, i.e. the total gradient magnitude from each segment's balancing term is equal regardless of the TransPoint position.

8.4 Full Training Loss

Combining all components, the total training loss is:

$$\boxed{\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda_a \mathcal{L}_{\text{bal}}^{(a)} + \lambda_s \mathcal{L}_{\text{bal}}^{(s)} + \lambda_{\text{sp}} \mathcal{L}_{\text{bal}}^{(\text{ssm-proj})} + \lambda_{\text{ffn}} \mathcal{L}_{\text{bal}}^{(\text{ffn})},} \tag{53}$$

where $\lambda_a = \alpha_0/\rho_\ell$, $\lambda_s = \alpha_0 \rho_\ell$, and $\lambda_{\text{sp}}, \lambda_{\text{ffn}}$ are set by grid search.

9 Design Problem 3: Training Stability

9.1 Statement of the Problem

Design Problem 9.1 (Training Stability under Coupled TransPoint and Routing)**.** TransMamba already has a fragile training regime: Table 6 of Li et al. [2026] shows that concentrated or insufficiently diverse TransPoint schedules cause validation loss degradation. MoE routing introduces a second source of instability: *expert collapse*, where the router assigns almost all tokens to a small subset of experts, making the remaining experts receive no gradient and stagnate. In TransMamba-MoE, these two instability sources are coupled: if the TransPoint changes the segment boundary, the effective routing distribution seen by each mechanism shifts, which can trigger expert collapse in the newly dominant segment. How can we design a training procedure that controls both forms of instability simultaneously?

9.2 Analysis of the Instability Sources

Source 1: TransPoint-induced routing distribution shift. Consider a change in the TransPoint from $\mathcal{P}_\ell$ to $\mathcal{P}'_\ell$ during training (which occurs when the TransPoint schedule changes between warmup and full training, or when fine-tuning with a different schedule). The attention segment changes from $[0, \mathcal{P}_\ell)$ to $[0, \mathcal{P}'_\ell)$; denoting $\Delta\mathcal{P} = \mathcal{P}'_\ell - \mathcal{P}_\ell$, the tokens in $[\mathcal{P}_\ell, \mathcal{P}'_\ell)$ switch from SSM-mode to attention-mode routing. Their routing distribution under the old W-MoE (tuned for SSM) is likely to be different from what the attention experts expect, inducing a gradient spike.

Source 2: Expert collapse dynamics. Let $\bar{f}_i$ be the time-averaged load fraction of expert i. Expert collapse occurs when $\bar{f}_i \to 0$ for some $i \neq i^*$ and $\bar{f}_{i^*} \to 1 - \sum_{j \neq i^*} \bar{f}_j \approx 1$. In the coupled system, the load-balancing loss $\mathcal{L}^{(a)}_{\text{bal}} + \mathcal{L}^{(s)}_{\text{bal}}$ must counteract collapse in both segments simultaneously.

9.3 Solution: Three-Component Stability Protocol

Solution 9.2 (Stability Protocol for TransMamba-MoE)**.** We propose three complementary mechanisms to ensure training stability.

Component 1: Gradual TransPoint Warm-Up. Rather than activating the full TransPoint schedule from epoch 1, we introduce a *warm-up schedule* $\mathbf{P}^{(r)}$ that linearly interpolates from a conservative schedule to the target schedule:

$$\mathcal{P}^{(r)}_\ell = T \cdot \big(1 - r/R\big) + \mathcal{P}_\ell \cdot (r/R), \quad r = 0, \ldots, R, \tag{54}$$

where r is the warm-up step and R is the total warm-up duration (measured in gradient steps). At $r = 0$, all layers are purely Transformer ($\mathcal{P}^{(0)}_\ell = T$); at $r = R$, the full TransPoint schedule is active. This ensures the routing distribution shifts gradually as the SSM segment grows, allowing the router to adapt without gradient spikes.

Lemma 9.3 (Gradient bound under warm-up)**.** *Let $\|\nabla_\phi \mathcal{L}_{\text{total}}\|^{(r)}$ denote the gradient norm of the router parameters at warm-up step r. Under the warm-up schedule (54), the*

change in gradient norm between consecutive steps satisfies:

$$\left\| \|\nabla_\phi \mathcal{L}_{\text{total}}\|^{(r+1)} - \|\nabla_\phi \mathcal{L}_{\text{total}}\|^{(r)} \right\| \leq \frac{C_{\text{grad}}}{R}, \tag{55}$$

for a constant C_{grad} depending on the Lipschitz constants of $\mathcal{L}_{\text{total}}$ and the router.

Proof. The gradient change between steps r and $r+1$ arises solely from the change $\Delta\mathcal{P}^{(r)} = \mathcal{P}_\ell^{(r+1)} - \mathcal{P}_\ell^{(r)} = (\mathcal{P}_\ell - T)/R$. Since $\mathcal{L}_{\text{total}}$ is Lipschitz continuous in $\mathcal{P}_\ell$ (the TransPoint enters as a discrete indicator, which we relax to a smooth sigmoid gate in the warm-up regime), the gradient change is at most $L_\mathcal{P} \cdot |\Delta\mathcal{P}^{(r)}| = L_\mathcal{P} \cdot |\mathcal{P}_\ell - T|/R$, which gives (55) with $C_{\text{grad}} = L_\mathcal{P}|\mathcal{P}_\ell - T|$. □

Component 2: Expert Auxiliary Noise for Collapse Prevention. Following Shazeer et al. [2017], we add *per-expert noise* to the router logits during training:

$$\tilde{\boldsymbol{\alpha}}_t = \boldsymbol{\alpha}_t + \boldsymbol{\epsilon}_t, \quad \boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \sigma_{\text{noise}}^2 I_{mN_e}), \tag{56}$$

where σ_{noise} decays during training:

$$\sigma_{\text{noise}}^{(r)} = \sigma_0 \cdot \exp(-r/\tau_{\text{noise}}), \tag{57}$$

with σ_0 and τ_{noise} as hyperparameters. The noise ensures all experts receive gradient signal early in training, preventing premature collapse.

Component 3: Segment-Decoupled Gradient Clipping. We apply separate gradient clipping to the router parameters ϕ within each segment:

$$g_\phi^{(a)} \leftarrow g_\phi^{(a)} \cdot \min\left(1, \frac{\eta_{\text{clip}}}{\left\|g_\phi^{(a)}\right\|}\right), \qquad dg_\phi^{(s)} \leftarrow g_\phi^{(s)} \cdot \min\left(1, \frac{\eta_{\text{clip}}}{\left\|g_\phi^{(s)}\right\|}\right), \tag{58}$$

where $g_\phi^{(a)}, g_\phi^{(s)}$ are the gradient contributions from the attention and SSM segments respectively, and η_{clip} is the clipping threshold. This prevents one segment's large gradient from dominating the routing update and inadvertently collapsing the other segment's expert distribution.

Theorem 9.4 (Stability of the three-component protocol)**.** *Under the warm-up schedule (54), the noise schedule (57), and the segment-decoupled gradient clipping (58), the expected gradient variance of the router parameters satisfies:*

$$\mathbb{E}\left[\|\nabla_\phi \mathcal{L}_{\text{total}}\|^2\right] \leq \frac{C_{\text{grad}}^2}{R^2} + \sigma_0^2 m N_e \cdot e^{-2r/\tau_{\text{noise}}} + \eta_{\text{clip}}^2, \tag{59}$$

and thus converges to η_{clip}^2 as $r \to \infty$ and $R \to \infty$.

Proof sketch. The three terms in (59) correspond to the three components: (1) the warm-up term decays as $1/R^2$ by Lemma 9.3; (2) the noise term is the variance of mN_e independent Gaussians with variance $\sigma_0^2 e^{-2r/\tau_{\text{noise}}}$ each, which decays exponentially; (3) the clipping term is the squared clipping threshold, which is the irreducible minimum gradient magnitude under our clipping scheme. All three terms vanish or stabilise, establishing bounded variance throughout training. □

10 The Complete TransMamba-MoE Layer

10.1 Full Forward Pass

We now assemble the complete single-layer forward pass of TransMamba-MoE. Let $H^{(\ell-1)} \in \mathbb{R}^{T\times d}$ be the input to layer ℓ.

Step 1: Layer normalisation.

$$\tilde{H} = \mathrm{RMSNorm}(H^{(\ell-1)}). \tag{60}$$

Step 2: Partition at TransPoint.

$$\tilde{H}_s = \tilde{H}[0:\mathcal{P}_\ell,\,:], \qquad d\tilde{H}_l = \tilde{H}[\mathcal{P}_\ell:T,\,:]. \tag{61}$$

Step 3: W-MoE routing (token-level, all T tokens). For each token position $t \in \{0,\dots,T-1\}$:

$$\boldsymbol{\alpha}_t = \tilde{h}_t \mathbf{W}_\phi, \tag{62}$$

$$\mathcal{E}_t = \text{top-k}(\mathrm{softmax}(\boldsymbol{\alpha}_t)[N_s+1:mN_e], K), \tag{63}$$

$$\mathbf{W}_t^* = \frac{1}{N_s}\sum_{i=1}^{N_s}\mathbf{W}^{(i)} + \sum_{i\in\mathcal{E}_t}\hat{s}_{t,i}\mathbf{W}^{(i)}. \tag{64}$$

Step 4: Shared projections via W-MoE. For $t < \mathcal{P}_\ell$ (attention segment):

$$Q_t = K_t = V_t = \delta(\tilde{h}_t^s \mathbf{W}_t^*). \tag{65}$$

For $t \geq \mathcal{P}_\ell$ (SSM segment, local index $k = t - \mathcal{P}_\ell$):

$$C_k = B_k = x_k = \delta(\tilde{h}_k^l \mathbf{W}_t^*). \tag{66}$$

Step 5: Attention output.

$$y_s = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V \in \mathbb{R}^{\mathcal{P}_\ell\times d}. \tag{67}$$

Step 6: SSM-MoE projections. For each local SSM position k:

$$\boldsymbol{\alpha}_k^{\mathrm{ssm}} = \tilde{h}_k^l \mathbf{W}_\phi^{\mathrm{ssm}}, \tag{68}$$

$$\Delta_k = \sigma(\tilde{h}_k^l \mathbf{W}_\Delta^{*,k} + b_\Delta), \tag{69}$$

$$A_k = \exp(-\Delta_k \circ e^{\log \mathbf{W}_A^{*,k}}), \tag{70}$$

$$Z_k = \delta(\tilde{h}_k^l \mathbf{W}_Z^{*,k}). \tag{71}$$

Step 7: Expert-Conditioned Memory Converter.

$$h_0^{\mathrm{ssm}} = \big(A^\times \circ \bar{K}^\top\big)\bar{V}\big|_{[-1]} + \sum_{t=0}^{\mathcal{P}_\ell-1}(A^\times)_{\mathcal{P}_\ell-1,t}\,\Phi_t, \tag{72}$$

with Φ_t as in (43).

Step 8: SSM recurrence.

$$h_k^{\text{ssm}} = A_{k-1} \odot h_{k-1}^{\text{ssm}} + B_k \circ \Delta_k \circ x_k, \tag{73}$$
$$(y_l)_k = C_k h_k^{\text{ssm}} \circ Z_k. \tag{74}$$

Note the gating by Z_k (element-wise product with the SSM output), which was implicit in the original TransMamba.

Step 9: Concatenate and residual.

$$y = [y_s;\, y_l] \in \mathbb{R}^{T\times d}, \qquad dH^{\text{mid}} = H^{(\ell-1)} + y. \tag{75}$$

Step 10: FFN-MoE sublayer.

$$H^{(\ell)} = H^{\text{mid}} + \text{FFN-MoE}\big(\text{RMSNorm}(H^{\text{mid}})\big). \tag{76}$$

10.2 Full Algorithm

Algorithm 1 TransMamba-MoE: Forward Pass for Layer ℓ

Require: $H^{(\ell-1)} \in \mathbb{R}^{T\times d}$, TransPoint $\mathcal{P}_\ell$, parameters $\boldsymbol{\theta}$.
Ensure: $H^{(\ell)} \in \mathbb{R}^{T\times d}$.

$\tilde{H} \leftarrow \text{RMSNorm}(H^{(\ell-1)})$
$\tilde{H}_s \leftarrow \tilde{H}[0:\mathcal{P}_\ell,:], \quad \tilde{H}_l \leftarrow \tilde{H}[\mathcal{P}_\ell:T,:]$
for $t = 0,\dots,T-1$ **do**
  $\boldsymbol{\alpha}_t \leftarrow \tilde{h}_t \mathbf{W}_\phi$ ▷ W-MoE router logits
  $\mathcal{E}_t \leftarrow \text{top-k}(\text{softmax}(\boldsymbol{\alpha}_t)[N_s+1:], K)$
  $\mathbf{W}_t^* \leftarrow \frac{1}{N_s}\sum_{i=1}^{N_s} \mathbf{W}^{(i)} + \sum_{i\in\mathcal{E}_t} \hat{s}_{t,i}\mathbf{W}^{(i)}$
end for
for $t = 0,\dots,\mathcal{P}_\ell - 1$ **do** ▷ Attention segment
  $Q_t, K_t, V_t \leftarrow \delta(\tilde{h}_t^s \mathbf{W}_t^*)$
end for
$y_s \leftarrow \text{softmax}(QK^\top/\sqrt{d})\,V$
Compute $\bar{K}, \bar{V}$ using $\bar{\mathbf{W}}$
$h_0^{\text{ssm}} \leftarrow (A^\times \circ \bar{K}^\top)\bar{V}|_{[-1]} + \sum_t (A^\times)_{\mathcal{P}_\ell-1,t}\Phi_t$ ▷ Expert-conditioned MCE, Def. 7.2
for $k = 0,\dots,T-\mathcal{P}_\ell-1$ **do** ▷ SSM segment
  $C_k, B_k, x_k \leftarrow \delta(\tilde{h}_k^l \mathbf{W}^*_{\mathcal{P}_\ell+k})$
  $\boldsymbol{\alpha}_k^{\text{ssm}} \leftarrow \tilde{h}_k^l \mathbf{W}_\phi^{\text{ssm}}$
  Compute Δ_k, A_k, Z_k via SSM-MoE, Eqs. (69)–(71)
  $h_k^{\text{ssm}} \leftarrow A_{k-1} \odot h_{k-1}^{\text{ssm}} + B_k \circ \Delta_k \circ x_k$
  $(y_l)_k \leftarrow C_k h_k^{\text{ssm}} \circ Z_k$
end for
$y \leftarrow [y_s;\, y_l]$
$H^{\text{mid}} \leftarrow H^{(\ell-1)} + y$
$H^{(\ell)} \leftarrow H^{\text{mid}} + \text{FFN-MoE}(\text{RMSNorm}(H^{\text{mid}}))$
return $H^{(\ell)}$

11 Parameter Count and FLOPs Analysis

11.1 Parameter Count

Table 1: Parameter counts of TransMamba-MoE vs. TransMamba [Li et al., 2026]. d: hidden dim; N: SSM state dim; m: fine-grain factor; N_e: expert groups; $N_s, N_s^{\text{ssm}}, N_s^{\text{ffn}}$: shared expert counts; $d_m = d_{\text{ffn}}/m$: expert intermediate dim; L: layers.

Component	**TransMamba**	**TransMamba-MoE**
W (shared weight)	d^2 per layer	$mN_e \cdot d^2$ per layer
$\mathbf{W}_\Delta, \mathbf{W}_A, \mathbf{W}_Z$	$2d^2 + dN$ per layer	$mN_e^{\text{ssm}}(2d^2 + dN)$ per layer
FFN	$2d \cdot d_{\text{ffn}}$ per layer	$mN_e^{\text{ffn}} \cdot 3dd_m$ per layer
Router $\mathbf{W}_\phi$	—	$d(mN_e + mN_e^{\text{ssm}} + mN_e^{\text{ffn}})$ per layer
Total (all layers)	$Ld^2(1 + 2 + d_{\text{ffn}}/d)$	$L\Big[mN_e d^2(2 + N/d) + mN_e^{\text{ffn}} 3dd_m\Big]$
Active per token	$d^2(3 + d_{\text{ffn}}/d)$	$(N_s + K)d^2(2 + N/d) + (N_s^{\text{ffn}} + K^{\text{ffn}})3dd_m$

11.2 FLOPs Per Layer

Table 2: FLOPs per layer. $\mathcal{P} = \mathcal{P}_\ell$; $\kappa = N_s + K$ (active W-MoE experts per token).

Operation	**TransMamba**	**TransMamba-MoE**
W-MoE routing	—	$T \cdot mN_e \cdot d$
Shared projection	Td^2	$T\kappa d^2$
Attention ($t < \mathcal{P}$)	$\mathcal{P}^2 d$	$\mathcal{P}^2 d$ (unchanged)
SSM-MoE routing	—	$(T - \mathcal{P})mN_e^{\text{ssm}} d$
SSM projection ($t \geq \mathcal{P}$)	$(T - \mathcal{P})Nd$	$(T - \mathcal{P})\kappa^{\text{ssm}} Nd$
Expert-cond. MCE	$\mathcal{P}dN$	$\mathcal{P}dN + \mathcal{P}d^2$
SSM recurrence	$(T - \mathcal{P})Nd$	$(T - \mathcal{P})Nd$
FFN-MoE	Td_{ffn}	$T(N_s^{\text{ffn}} + K^{\text{ffn}})dd_m$
Total	$\mathcal{O}(\mathcal{P}^2 N + (T - \mathcal{P})N^2)$	$\mathcal{O}(\mathcal{P}^2 N + (T - \mathcal{P})\kappa^{\text{ssm}} Nd + T\kappa d^2)$

12 Implementation Contract

This section translates the mathematical specification into a set of unambiguous implementation requirements for the coding phase.

12.1 Data Structures

I1. Expert pool tensors. The W-MoE pool is stored as a single tensor `W_pool` $\in \mathbb{R}^{mN_e \times d \times d}$, where `W_pool`$[i]$ is $\mathbf{W}^{(i)}$. The effective weight $\mathbf{W}_t^*$ is computed via `torch.einsum` or `torch.matmul` with the routing score vector as the coefficient, *without* materialising the full $T \times d \times d$ tensor.

I2. Routing decision cache. For each token t and layer ℓ, store $\mathcal{E}_t$ (as an integer index tensor of shape (T, K)) and $\hat{s}_{t,\cdot}$ (as a float tensor of shape (T, K)), so that the MCE correction term Φ_t (43) can access the routing decomposition without recomputation.

I3. Segment masks. Maintain Boolean tensors `attn_mask` $\in \{0,1\}^T$ and `ssm_mask` $\in \{0,1\}^T$ (complement of each other, split at $\mathcal{P}_\ell$), used to index into H for computing segment-specific load statistics (46)–(47).

12.2 Critical Implementation Invariants

C1. Coupling invariant. For any token t, the routing decision $\mathcal{E}_t$ used to compute Q_t, K_t, V_t (65) must be identical to that used to compute C_k, B_k, x_k (66) for the same token (whether it appears in the attention or SSM segment). Violation breaks the MCE derivation.

C2. Norm tracking. Since amplitude encoding of $\mathbf{W}_t^*$ discards the norm (all three projections Q, K, V are numerically identical), the downstream code must *not* rely on $Q \neq K \neq V$; instead, the three outputs are differentiated by their position in the attention formula (67).

C3. Gradient checkpointing. The MCE correction (43) requires R_t and $\bar{K}_t, \bar{V}_t$ for all attention-segment tokens simultaneously. Under gradient checkpointing, these must be recomputed in the backward pass, not stored. The implementation should checkpoint at the MCE boundary.

C4. TransPoint warm-up interface. The TransPoint $\mathcal{P}_\ell$ must be a configurable runtime parameter (not a compile-time constant), passed into the layer as a tensor of shape $(L,)$, so that the warm-up schedule (54) can modify it across training steps.

C5. Load balancing accumulation. The segment-specific load statistics $f_i^{(a)}, f_i^{(s)}, P_i^{(a)}, P_i^{(s)}$ must be accumulated across all tokens *within each batch* before computing the loss, not computed per-token and averaged. Computing them per-token introduces a discrete-selection bias.

12.3 Suggested Hyperparameters for First Run

Table 3: Recommended hyperparameters for the first training run of TransMamba-MoE, based on the 1.5B TransMamba settings of Li et al. [2026] (Table 7–8 therein) and DeepSeekMoE settings of Dai et al. [2024].

Hyperparameter	**Symbol**	**Suggested Value**
Expert groups	N_e	8
Fine-grain factor	m	4 (total: 32 experts)
Shared W-MoE experts	N_s	2
Active routed W-MoE	K	4
SSM expert groups	N_e^{ssm}	4
SSM shared experts	N_s^{ssm}	1
SSM active routed	K^{ssm}	2
FFN expert groups	N_e^{ffn}	8
FFN shared experts	N_s^{ffn}	2
FFN active routed	K^{ffn}	2
Balance coefficient	α_0	1×10^{-2}
Router noise σ_0	—	0.1
Noise decay τ_{noise}	—	5,000 steps
Grad clip threshold	η_{clip}	1.0
Warm-up steps	R	2,000
TransPoint schedule	$\mathbf{P}$	$\{0, 128, 256, 512, 1024, 2048, 4096, 8192\}$ cycled every 8 layers

13 Open Theoretical Questions

This section records theoretical questions that are open at the time of this working draft and should be addressed before moving to an ablation study.

Q1. Tightness of the MCE error bound. Theorem 7.3 gives an $\mathcal{O}(\|R_t\|_F^2)$ bound on the MCE approximation error. Is this bound tight, or can the exact MCE (Definition 7.2) be computed without any approximation? We conjecture the latter is true, but a formal proof is needed.

Q2. Optimal segment balance coefficients. The heuristic (52) for α_a, α_s is derived from a segment-length argument. A deeper analysis should characterise the optimal (α_a, α_s) as a function of the routing entropy in each segment.

Q3. Expert specialisation under the TransPoint schedule. Does the fine-grained TransPoint schedule of Li et al. [2026] induce systematic expert specialisation across layers (i.e. do SSM-segment experts at early layers learn different representations than those at late layers)? This question is empirically testable but has theoretical implications for the depth of the MoE architecture.

Q4. Inconsistent training/inference scheduling with MoE. Section 3.4 of Li et al. [2026] shows that TransMamba can be inferred with a different TransPoint schedule than it was trained on. In TransMamba-MoE, does this carry over? The MCE correction term (43) depends on $\mathcal{P}_\ell$ explicitly; changing this at inference modifies the correction, and the routed experts may not generalise.

14 Conclusion

This working draft establishes the complete mathematical foundation for TransMamba-MoE. Starting from an explicit derivation of all notation, we have:

(i) Defined the W-MoE pool with the coupling constraint ensuring the shared-parameter duality of Li et al. [2026] is preserved across all expert selections.

(ii) Defined the SSM-MoE pool with shared routing across the Δ, A, and Z projections.

(iii) Defined the FFN-MoE following DeepSeekMoE [Dai et al., 2024].

(iv) Solved Design Problem 1 by deriving the Expert-Conditioned Memory Converter (Definition 7.2) with an explicit error bound (Theorem 7.3).

(v) Solved Design Problem 2 by deriving mechanism-aware segment-specific load-balancing losses with a theoretically grounded coefficient heuristic (Theorem 8.4).

(vi) Solved Design Problem 3 by proposing a three-component stability protocol with proven gradient variance bounds (Theorem 9.4).

The complete layer forward pass is given in Algorithm 1, and the implementation contract in Section 12 translates every equation into a concrete coding requirement. This document is intended to be the sole reference during the implementation phase: no design decision should be made during coding that is not traceable to a numbered equation or theorem here.

References

D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.

A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

A. Katharopoulos et al. Routing Mamba: Sparse mixture of experts for SSM-based language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Y. Li, R. Xie, Z. Yang, X. Sun, S. Li, W. Han, Z. Kang, Y. Cheng, C. Xu, D. Wang, and J. Jiang. TransMamba: A sequence-level hybrid Transformer-Mamba language model. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. URL https://arxiv.org/abs/2503.24067. arXiv:2503.24067v2 [cs.LG], accepted AAAI 2026.

N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *International Conference on Learning Representations (ICLR)*, 2017.